🚀 OBJECT ORIENTED SYSTEMS (OOS) — 2022 QUESTION 4 COMPLETE DETAILED SOLUTION

🌟 Introduction

Question 4 of the 2022 Object Oriented Systems examination belongs to CO3 and focuses on some of the most advanced paradigms of Java programming such as **Reflection**, **Generic Programming**, **Constructor Overloading**, and **Constructor Chaining**. These concepts are extremely important not only from the university examination perspective but also from the viewpoint of real-world software engineering because modern Java frameworks like Spring, Hibernate, and JUnit heavily depend upon these advanced mechanisms internally.

Unlike elementary Java programs, these topics demonstrate how Java supports:

- Runtime introspection
- Dynamic method invocation
- Reusable programming structures
- Highly scalable software design
- Flexible object-oriented architecture

Therefore, understanding these concepts deeply helps a student move from basic Java programming toward enterprise-level Java development 🚀☕.

📃 QUESTION 4(a)

❓ **Write suitable code snippet to invoke the `length()`, `charAt()` and `indexOf()` methods of String class using reflection. Illustrate their functionalities with suitable examples.**

✅ Answer

One of the most powerful capabilities provided by Java is the concept of **Reflection**. Reflection is a mechanism through which a Java program can inspect and manipulate its own structure during runtime. Normally, in traditional programming, method calls are fixed during compilation. However, reflection allows a program to dynamically discover and invoke methods during execution. This capability provides Java with tremendous flexibility and makes advanced frameworks possible.

In simple words, reflection enables a program to:

- Obtain information about classes
- Inspect methods and constructors
- Access private members
- Dynamically invoke methods
- Even modify fields during runtime

This process is also known as *Introspection* because the program "looks into itself" during execution 🧠.

The Reflection API is available inside the `java.lang.reflect` package.

🌟 Why Reflection is Needed?

Reflection is heavily used in:

- Spring Framework
- Hibernate ORM
- IDEs
- Annotation processors
- Dependency Injection systems
- Unit testing frameworks

For example, when Spring automatically creates objects and injects dependencies without explicitly calling constructors, reflection is working internally behind the scenes 🚀.

🌟 Invoking String Methods Using Reflection

Suppose we have a `String` object and we want to invoke `length()`, `charAt()`, and `indexOf()` dynamically during runtime rather than directly writing `str.length();`. This can be done using reflection.

✅ Complete Java Program

```
import java.lang.reflect.*;

class ReflectionDemo {
    public static void main(String[] args) throws Exception {
        String str = "OBJECT";

        // Obtaining Class object
        Class c = str.getClass();

        // Accessing length() method
        Method m1 = c.getMethod("length");
        Object len = m1.invoke(str);
        System.out.println("Length = " + len);

        // Accessing charAt() method
        Method m2 = c.getMethod("charAt", int.class);
        Object ch = m2.invoke(str, 2);
        System.out.println("Character = " + ch);

        // Accessing indexOf() method
        Method m3 = c.getMethod("indexOf", String.class);
        Object pos = m3.invoke(str, "EC");
        System.out.println("Index = " + pos);
    }
}
```

🖥️ Output

```
Length = 6
Character = J
Index = 3
```

🌟 Deep Explanation of Program Flow

Let us now understand the internal working of the program step-by-step because this is the most important part from an examination perspective 🧠.

🔹 Step 1 — Creating String Object

```
String str = "OBJECT";
```

Here a `String` object named `str` is created.

🔹 Step 2 — Obtaining Class Metadata

```
Class c = str.getClass();
```

Every object in Java internally carries metadata about its class. The method `getClass()` returns a `Class` object containing:

- Class name
- Method details
- Constructor information
- Field information

Thus the variable `c` now stores metadata of the `String` class.

🔹 Step 3 — Obtaining Method Object

```
Method m1 = c.getMethod("length");
```

Here reflection searches inside the `String` class and retrieves information about the method named `length()`. This information is stored inside a `Method` object.

🔹 Step 4 — Dynamic Invocation

```
m1.invoke(str);
```

This statement dynamically executes `str.length();` during runtime. The returned value becomes `6` because `"OBJECT"` contains 6 characters.

🌟 Understanding `charAt()`

The method `charAt(index)` returns the character located at a specified index. Example: `"OBJECT".charAt(2)` returns `J` because indexing starts from zero.

Index	0	1	2	3	4	5
Character	O	B	J	E	C	T

Thus index `2` corresponds to `J`.

🌟 Understanding `indexOf()`

The method `indexOf()` returns the starting position of a substring. Example: `"OBJECT".indexOf("EC")` returns `3` because the substring `"EC"` starts at index 3.

🌟 Advantages of Reflection

Reflection provides several important advantages in Java development 🚀:

- **Dynamic method invocation:** Methods can be discovered and executed during runtime rather than being fixed during compilation.
- **Flexible framework development:** Modern enterprise frameworks depend upon reflection extensively.
- **Advanced capabilities:** Reflection enables automatic dependency injection, dynamic object creation, and annotation processing.
- **Tooling support:** It helps in debugging tools, IDE design, and testing frameworks.

⚠️ Drawbacks of Reflection

Although powerful, reflection also has certain disadvantages ⚠️:

- **Performance overhead:** Reflection is slower than direct method invocation because additional runtime inspection is involved.
- **Breaks encapsulation:** It can break encapsulation by accessing `private` members.
- **Security risks:** It may create security concerns because runtime access restrictions can be bypassed.

Thus reflection should be used carefully.

🎯 Conclusion

Reflection is one of the most advanced and powerful features of Java. It allows programs to inspect and manipulate classes dynamically during runtime. By using reflection, methods such as `length()`, `charAt()`, and `indexOf()` can be invoked dynamically, making Java highly flexible and framework-oriented 🚀.

📄 QUESTION 4(b)

❓ Use suitable code snippet to show how to access/invoke the forbidden methods of a class using reflection. Also show how a final field of a class can be modified.

✅ Answer

One of the most interesting and powerful capabilities of Java Reflection is that it can bypass normal access control mechanisms of Java. Normally, Java follows strict encapsulation principles and prevents outside access to:

- `private` methods
- `private` variables
- `final` fields

However, reflection provides the capability to override these restrictions dynamically during runtime ⚠️. This is achieved using the method:

```
setAccessible(true)
```

which disables Java access checking temporarily. Although this feature is extremely powerful, it is also dangerous because it can violate object-oriented security and encapsulation principles.

🌟 Accessing Private Methods Using Reflection

Suppose a class contains a `private` method. Normally that method cannot be invoked from outside the class. However, reflection can access and invoke it dynamically.

✅ Program to Invoke Private Method

```
import java.lang.reflect.*;

class Secret {
    private void hidden() {
        System.out.println("Private Method Invoked");
    }
}

class Demo {
    public static void main(String[] args) throws Exception {
        Secret obj = new Secret();
        Class c = obj.getClass();

        Method m = c.getDeclaredMethod("hidden");

        // Bypassing access control
        m.setAccessible(true);
        m.invoke(obj);
```

```
    }
}
```

🖥️ Output

```
Private Method Invoked
```

🌟 Deep Explanation

- Initially the method `hidden()` is `private` and inaccessible outside the class.
- The statement `getDeclaredMethod("hidden")` retrieves metadata of the private method.
- Then `m.setAccessible(true);` disables Java's normal access checking.
- Finally `m.invoke(obj);` executes the method dynamically during runtime.

Thus reflection bypasses encapsulation restrictions 🚀.

🌟 Modifying Final Fields Using Reflection

Normally `final` variables cannot be modified after initialization because they are designed to provide immutability. However, reflection can even modify `final` fields.

✅ Program to Modify Final Field

```
import java.lang.reflect.*;

class Sample {
    private final int x = 10;

    void show() {
        System.out.println("x = " + x);
    }
}

class Demo {
    public static void main(String[] args) throws Exception {
        Sample obj = new Sample();
        Class c = obj.getClass();

        Field f = c.getDeclaredField("x");

        // Bypassing access control
        f.setAccessible(true);
        f.set(obj, 100);

        obj.show();
```

```
    }
}
```

🖥️ **Output**

```
x = 100
```

🌟 Why This is Dangerous?

This operation violates immutability principles because a supposedly constant field is being changed dynamically ⚠️. Such modifications can:

- Break program consistency
- Create unpredictable behavior
- Violate security assumptions

Therefore, reflection should be used responsibly.

🌟 Real-World Uses of Reflection

Reflection is widely used in:

- Spring Framework
- Hibernate
- JUnit
- Dependency Injection
- Serialization systems
- IDE development tools

These frameworks dynamically inspect and manipulate classes during runtime using reflection.

🎯 Conclusion

Reflection can dynamically access forbidden methods and modify final fields by bypassing Java access restrictions. Although this makes Java extremely powerful and flexible, careless use of reflection can compromise encapsulation and security principles 🚀.